Manual técnico — PromptZal 3

Manual técnico — PromptZal 3

1. Arquitectura

Aplicación Java Swing organizada por fases: Lexer convierte el texto en tokens; Parser implementa descenso recursivo manual; Modelo contiene el AST; Compilador coordina semántica, generación de prompt y reportes; Main ofrece la interfaz.

2. Terminales

DIRECTIVA (@modelo, @rol, @formato), AGENTE, CONTEXTO, VARIABLE, EJECUTAR, EXPORTAR, COMANDO_IA, CARGAR, CONECTOR (SOBRE/DESDE/EN/COMO), FLECHA, IGUAL, MAS, LLAVE_A, LLAVE_C, PAR_A, PAR_C, COMA, ID, CADENA, NUMERO, EOF. Los errores léxicos incluyen carácter desconocido, cadena sin cerrar y comentario de bloque sin cerrar. Filas y columnas comienzan en 1.

3. Gramática BNF

```
<programa> ::= <directivas> <agentes> <ejecuciones> <exportar> EOF
<directivas> ::= DIRECTIVA CADENA <directivas> | ε
<agentes> ::= <agente> <mas_agentes>
<mas_agentes> ::= <agente> <mas_agentes> | ε
<agente> ::= AGENTE ID LLAVE_A <contexto> <variables> <comandos> LLAVE_C
<contexto> ::= CONTEXTO IGUAL CADENA
<variables> ::= VARIABLE ID IGUAL <expresion> <variables> | ε
<comandos> ::= <comando> <mas_comandos>
<mas_comandos> ::= <comando> <mas_comandos> | ε
<comando> ::= COMANDO_IA <expresion> <conectores> FLECHA ID
<conectores> ::= CONECTOR <expresion> <conectores> | ε
<expresion> ::= <termino> <mas_terminos>
<mas_terminos> ::= MAS <termino> <mas_terminos> | ε
<termino> ::= CADENA | ID | NUMERO <unidad> | <cargar>
<unidad> ::= ID | ε
<cargar> ::= CARGAR PAR_A CADENA PAR_C
<ejecuciones> ::= EJECUTAR ID <mas_ejecuciones>
<mas_ejecuciones> ::= EJECUTAR ID <mas_ejecuciones> | ε
<exportar> ::= EXPORTAR ID <mas_ids>
<mas_ids> ::= COMA ID <mas_ids> | ε
```

4. Lexer JFlex

Las reglas están en src/main/jflex/PromptZalLexer.flex. Define palabras reservadas, directivas, identificadores, números, cadenas, operadores, delimitadores y comentarios. El archivo Lexer.java incluido es una implementación funcional autocontenida para ejecutar el proyecto sin generar fuentes externas. Para la entrega final, se recomienda regenerar con JFlex en el entorno del curso y conectar el lexer generado al contrato Token.

5. Parser descendente recursivo

Cada grupo de producciones tiene métodos identificables: directivas, agentes, agente, comando, expresion, termino, ejecuciones y exportar. No se usa CUP, JavaCC, ANTLR ni otro generador de parsers. Ante una discrepancia se registra token encontrado/esperado, posición y se intenta recuperación en los tokens AGENTE, LLAVE_C, COMANDO_IA, VARIABLE, EJECUTAR y EXPORTAR.

6. AST y tabla de símbolos

El AST representa directivas, agentes, contexto, variables, comandos, expresiones, ejecuciones y exportaciones. La tabla de símbolos contiene nombre, tipo (agente, variable declarada o resultado de comando), ámbito y línea de declaración.

7. Semántica

Se detectan agentes duplicados, variables referenciadas antes de declararse, agentes inexistentes en EJECUTAR y variables no disponibles para EXPORTAR en agentes ejecutados. GENERAR puede recibir un identificador como tipo de contenido y las unidades tras números (por ejemplo 100 palabras) no son variables.

8. Generación y reportes

El generador recorre el AST y redacta encabezado, agentes ejecutados, variables y sus orígenes, instrucciones en orden y resultados exportados. Se pueden guardar prompt .md/.txt, reporte HTML de errores, tabla de símbolos HTML y AST indentado HTML.

9. Limitaciones conocidas y mejoras

La interfaz suministrada es equivalente nueva porque el código del Proyecto 1 no venía adjunto; no puede afirmarse que lo reutiliza literalmente. El archivo .flex debe probarse/regenerarse con la versión de JFlex del laboratorio antes de presentar como evidencia de generación. Conviene ampliar pruebas de recuperación para errores en bloques anidados y enriquecer el tratamiento específico de conectores para todos los comandos.